Spotter

EECS 3311 Stage 2 Implementation Report

Dev Parikh, 219957794
EECS 3311 Software Design, Fall 2026
Repository: https://github.com/devparikh28/EECS-3311-Project--Spotter
Submitted 10 November 2026

Contents

1. Project overview and implementation status
2. Feature to implementation mapping
3. Implemented design patterns
4. Changes and rationale
5. AI tools and collaboration log
6. Development and verification summary
7. Demonstration index

---

1. Project overview and implementation status

What the application is

Spotter is an AI strength training and nutrition coach. It imports a lifter's training history and recovery data, computes strength metrics deterministically, and uses an LLM agent to plan training blocks, adapt sessions to recovery, substitute exercises the lifter cannot perform, plan competition attempts, suggest meals against remaining macros, and answer coaching questions.

The intended users are strength trainees generally rather than competitive powerlifters only. The profile carries a training goal of meet preparation, strength, hypertrophy or general fitness, and a meet date is optional.

The principle the design rests on

The model proposes and deterministic code disposes.

Every figure a lifter trains off is calculated rather than generated: estimated maxima, the RPE chart, attempt percentages, macro arithmetic, recovery rules and plan adherence are all ordinary Java and all work with no API key configured. Where a model is involved, it returns a proposal which `ResponseParser` turns into domain objects and `PlanValidator` either accepts or refuses. A refused proposal goes back to the model with the reason attached so it can correct itself. No model output reaches the domain model without passing both.

This is why section 2 classifies the AI features as hybrid rather than AI, even though Stage 1 labelled them AI. The classification changed because the implementation made the gate explicit.

AI and LLM integration

Spotter calls the Anthropic API through LangChain4j. Exactly one class, `agent/ClaudeClient`, imports that library; everything else depends on the `LLMClient` interface. Two models are configured and selected per agent rather than globally: `claude-haiku-4-5-20251001` for agents that pick from a filtered list or write one checked sentence, and `claude-sonnet-4-6` for agents that write or rewrite a whole training block or interpret prose.

The agent behaviour is not prompt forwarding. `CoachAgent` is a Template Method whose `run()` is final and which implements a tool loop, a validation gate and a retry: the agent gathers context, offers the model a toolset, lets it call tools for up to six rounds, parses the reply, validates it, and on refusal sends the reason back for up to two further attempts. Seven tools exist for the model to call, including `allowed_load`, which lets it ask what load a progression permits rather than guessing.

Implementation status

All fourteen features specified in Stage 1 are implemented and reachable in the submitted application. All eight design patterns named in Stage 1 are implemented, against a course minimum of five.

Measure	Value
Features implemented	14 of 14 from Stage 1, plus 4 new features (N01 to N04)
Distinct design patterns	8, plus Builder and Repository as supporting patterns
Main source files	183
Test source files	47
Automated tests	510, all passing
Measured API cost, whole project	about $2.9

Interfaces

Both interfaces required by Stage 1 are present. The JavaFX dashboard has ten screens: Dashboard, Today, Plan, Analytics, Recovery, Nutrition, Coach and Reports as tabs, with Import and Profile beside them. It also has a quick coach drawer on every screen, light and dark themes, PDF and CSV report export, and a captioned guided tour of every screen. The picocli command line has fifteen commands under the name `spotter`. Nine of the fifteen never call a model, which is a design outcome rather than a shortfall: the figures a lifter trains off keep working when the network does not.

Limitations and incomplete functionality, stated honestly

F09 has no command line route. Macro targets and food logging are reachable from the Nutrition screen only. Logging food from a terminal is a worse experience than the panel and would have been built for completeness rather than use.

F13 was built differently from its design. Stage 1 specified an adaptive interview of up to eight questions. The implementation reads a single paragraph and reports what it understood alongside what the lifter has not said. The reasoning is in section 4.

F14 has no offline demonstration mode. Every other AI feature has a prepared reply so it can be shown without an API key. Reading an arbitrary programme document is not something keyword matching can fake, so the command says so outright instead of pretending.

A known domain defect is recorded and not fixed. A competition lift variation such as `Bench Press - Close Grip (Barbell)` maps to `Lift.BENCH`, so the validator holds it to competition bench loads and refuses loads that are correct for the variation. The fix is to map only the competition movement itself and leave variations to judgement, as accessories already are. It was recorded rather than made because it touches how the catalogue marks competition lifts, which also feeds the estimated maxima in `StrengthAnalytics`, and a change with two consumers deserves more care than the last week of a project allows. Section 4, change 13.

The undo stack does not survive a restart. It lives in the window and is bounded at twenty edits. Section 4, change 14.

Stage 3 work has not started. The `--json` output on eight commands was built for the KUMA harness, but `tools/kuma/` does not exist yet.

---

2. Feature to implementation mapping

Every Stage 1 feature is accounted for below. No features were added beyond the fourteen Stage 1 specified, and none were removed.

On classification. Stage 1 labelled F05, F06, F07, F10, F11, F13 and F14 as AI, F08 as hybrid, and the rest as deterministic. This report labels every feature that calls a model as hybrid, because in the implementation none of them returns model output directly: each one passes through `ResponseParser` and a `PlanValidator` method before anything reaches the domain. That is a change in description rather than in intent, and it is the design decision the whole project is built around.

F01 — Lifter Profile and Constraints

Field	Entry
Stage 1 reference	F01
Status	Implemented
Classification	Deterministic
Implemented classes	`domain/LifterProfile` with its nested `Builder`, `domain/Limitation`, `domain/DietaryConstraints`, `domain/Equipment`, `domain/ExperienceLevel`, `domain/TrainingGoal`, `domain/SplitPreference`, `domain/WeightUnit`, `infrastructure/persistence/ProfileRepository`, `SqliteProfileRepository`, `presentation/gui/panels/ProfilePanel`
Key methods	`LifterProfile.canPerform(Exercise)`, `hasEquipment(Equipment)`, `weeksUntilMeet(LocalDate)`, `LifterProfile.Builder.build()`, `ProfileRepository.save(LifterProfile)`, `ProfileRepository.current()`
Design patterns	Builder, Repository
Video	GUI demo, segment F01

How the classes collaborate. `ProfilePanel` collects the lifter's bodyweight, goal, experience, training days, equipment and limitations and assembles them through `LifterProfile.Builder`, which validates as it goes rather than producing a half built profile. `SqliteProfileRepository.save` writes the profile across four tables, because equipment and limitations are lists and a limitation carries lists of its own; saving rewrites the children rather than reconciling them. The profile is then read by almost everything else: `canPerform` is what stops the substitution agent proposing a movement the lifter has no equipment for, and `weeksUntilMeet` is what lets block generation size a block to a competition date.

`SqliteProfileRepositoryTest` covers the failure this shape of code invites, which is saving twice and accumulating duplicate equipment.

F02 — Workout Logging and Import

Field	Entry
Stage 1 reference	F02
Status	Implemented
Classification	Deterministic
Implemented classes	`infrastructure/csv/DataSource`, `HevyCsvAdapter`, `GenericCsvAdapter`, `ColumnMapping`, `CsvFileReader`, `CsvImportService`, `DataSourceFactory`, `infrastructure/persistence/ImportProfileRepository`, `SqliteImportProfileRepository`, `WorkoutRepository`, `SqliteWorkoutRepository`, `presentation/cli/ImportCommand`, `LogCommand`
Key methods	`CsvImportService.importFile(Path)`, `importMapped(Path, ColumnMapping)`, `DataSourceFactory.detect(Path)`, `create(SourceKind)`, `createMapped(ColumnMapping)`, `ColumnMapping.suggestFor(List)`, `validate(List)`, `columnFor(String)`, `DataSource.read(Path)`, `skipped()`
Design patterns	Adapter, Factory Method, Repository
Video	CLI demo (deterministic), segment F02

How the classes collaborate. `ImportCommand` hands the file to `CsvImportService`, which asks `DataSourceFactory.detect` what the file is. The factory reads the header through `CsvFileReader` and returns the matching `DataSource` implementation: `HevyCsvAdapter` for a Hevy export, or `GenericCsvAdapter` carrying a `ColumnMapping` for an export from an application Spotter does not know. The adapter converts rows into `WorkoutSession` and `SetEntry` objects, resolving exercise names through `ExerciseCatalog`, and reports the rows it could not parse through `skipped()` rather than throwing. `CsvImportService` removes duplicates and returns an `ImportReport`.

Error and alternative behaviour. The adapter reads a real 7318 row Hevy export and skips 31 rows, reporting why for each. One unreadable row does not lose the other 7287. A mapping that names a column the file does not have is refused before anything is read, because the alternative is importing nothing and reporting success, which is the worst outcome available. Re importing the same file is detected and skipped rather than duplicating the history.

Four wrong assumptions in the Stage 1 design were corrected by meeting a real export, in commit `dd73edb`.

`spotter log` records training without any export: one movement typed in, with the closest catalogue names offered when the spelling is wrong, or `--as-prescribed`, which records today's planned session as performed.

F03 — Recovery Input

Field	Entry
Stage 1 reference	F03

Field	Entry
Status	Implemented
Classification	Deterministic
Implemented classes	`infrastructure/csv/WhoopCsvAdapter`, `domain/RecoveryDay` with its nested `Builder`, `domain/RecoverySource`, `domain/RecoveryRuleEngine`, `domain/RecoveryFlag`, `infrastructure/persistence/RecoveryRepository`, `SqliteRecoveryRepository`, `presentation/cli/RecoveryCommand`, `CheckInCommand`, `presentation/gui/panels/RecoveryPanel`
Key methods	`WhoopCsvAdapter.read(Path)`, `RecoveryDay.on(LocalDate, RecoverySource)`, `RecoveryRepository.findSince(LocalDate)`, `RecoveryRuleEngine.evaluate(RecoveryDay)`
Design patterns	Adapter, Builder, Repository
Video	CLI demo (deterministic), segment F03

How the classes collaborate. `WhoopCsvAdapter` parses a physiological cycles export, 216 rows in the file tested, keeping the longest sleep cycle per date because the export contains naps. Each row becomes a `RecoveryDay` through its builder. `CheckInCommand` produces the same object from three typed numbers with `RecoverySource.MANUAL_CHECK_IN`, then calls `RecoveryRuleEngine.evaluate` and prints the verdict immediately.

Why the manual route exists. Without it the whole recovery feature was conditional on owning a particular wearable: a lifter without a Whoop could log months of training and never have a session questioned. It is deliberately three numbers rather than a questionnaire, because anything longer gets skipped on exactly the mornings it matters.

F04 — Strength Analytics

Field	Entry
Stage 1 reference	F04
Status	Implemented
Classification	Deterministic
Implemented classes	`domain/StrengthAnalytics`, `domain/RPEChart`, `domain/AnalyticsSummary`, `domain/Point`, `domain/Range`, `presentation/cli/AnalyticsCommand`, `presentation/gui/panels/AnalyticsPanel`
Key methods	`StrengthAnalytics.computeE1RM(SetEntry)`, `canEstimate(SetEntry)`, `e1rmSeries(Lift, List)`, `weeklyTonnage(Lift, List)`, `bestSet(Lift, List)`, `currentE1RMs(List)`, `trend(Lift, List)`, `RPEChart.percentOf1RM(int, double)`
Design patterns	None specific; this is the deterministic core the agents read through `AnalyticsTool`
Video	CLI demo (deterministic) segment F04, GUI demo segment F04 for the chart

How the classes collaborate. `StrengthAnalytics.computeE1RM` applies `RPEChart.percentOf1RM` to a logged set to estimate a one rep maximum, aggregates estimates by week into a `List<Point>`, and returns

everything the screen plots inside an `AnalyticsSummary`.

Error and alternative behaviour. The RPE chart is a lookup table rather than a formula and refuses combinations outside it rather than extrapolating, so a set at RPE 10 for eight reps produces a data quality warning rather than a confident wrong number. No history produces an empty state rather than an error. 34 tests cover this across `RPEChartTest` and `StrengthAnalyticsTest`.

No model is involved anywhere in this feature, and `spotter analytics squat` runs with no API key configured.

F05 — Training Block Generation

Field	Entry
Stage 1 reference	F05
Status	Implemented
Classification	Hybrid. The model proposes a block; deterministic code validates every load before it can be saved
Implemented classes	`agent/agents/BlockGenerationAgent`, `agent/CoachAgent`, `PromptBuilder`, `ResponseParser`, `PlanValidator`, `ToolManager`, `agent/tools/AnalyticsTool`, `ExerciseDBTool`, `LoadRangeTool`, `domain/ProgressionStrategy`, `AbstractProgression` and four implementations, `ProgressionStrategyFactory`, `domain/TrainingBlock`, `Week`, `Session`, `ExercisePrescription`, `infrastructure/persistence/BlockRepository`, `SqliteBlockRepository`, `presentation/cli/PlanCommand`, `presentation/gui/panels/PlanPanel`
Key methods	`CoachAgent.run()` (final), `BlockGenerationAgent.createToolset()`, `systemPrompt()`, `parse(String)`, `validate(TrainingBlock)`, `ResponseParser.toBlock(JsonNode, LocalDate)`, `PlanValidator.validateBlock(...)`, `ProgressionStrategyFactory.forGoal(TrainingGoal, ExperienceLevel)`, `ProgressionStrategy.loadRange(...)`
Design patterns	Template Method, Strategy, Factory Method
Video	CLI demo (agent), segment F05; GUI demo for the Plan screen

How the classes collaborate. `PlanCommand` or `PlanPanel` builds a `BlockGenerationAgent` and calls the inherited `run()`. `CoachAgent.run()` is final and drives the whole sequence: it calls `gatherContext()` for the lifter's profile and current maxima, `createToolset()` for the tools this agent may use, and `PromptBuilder.build` for the request. The model may call `AnalyticsTool` for history, `ExerciseDBTool` for movements the lifter can perform, and `LoadRangeTool` to ask what load the progression allows, for up to six rounds. `ResponseParser.toBlock` turns the JSON reply into a `TrainingBlock` of `Week`, `Session` and `ExercisePrescription`, resolving every exercise name through the catalogue. `PlanValidator.validateBlock` then checks each prescribed load against the `Range` that the `ProgressionStrategy` selected by `ProgressionStrategyFactory.forGoal` permits.

Error and alternative behaviour, and the single best demonstration of the design. Verified against the live model on 2 November: a four week block for three lifts was refused on the first attempt because four close grip bench loads fell outside the allowed range. The reason went back to the model, which called

`allowed_load`, corrected the four loads, and the second attempt was accepted. Nine model calls, 18,443 tokens in and 12,460 out. Nothing was written to the database until the lifter pressed keep.

`LoadRangeTool` was added during that session precisely because the model was guessing at the allowed range and being refused for it. Section 4, change 3.

F06 — Equipment Substitution

Field	Entry
Stage 1 reference	F06
Status	Implemented
Classification	Hybrid. The candidate list is filtered deterministically; the model chooses from it and explains the choice; the choice is then checked
Implemented classes	`agent/agents/SubstitutionAgent`, `domain/Substitution`, `domain/SubstitutionReason`, `agent/tools/ExerciseDBTool`, `infrastructure/catalog/ExerciseCatalog`, `presentation/cli/SubstituteCommand`, `application/plan/SwapExerciseCommand`, substitution controls in `presentation/gui/panels/TodayPanel`
Key methods	`SubstitutionAgent.validate(Substitution)`, `ResponseParser.toSubstitution(...)`, `Substitution.trainsSomethingSimilar()`, `loadRatio()`, `apply()`, `ExerciseCatalog.performable(Set)`, `SwapExerciseCommand.describe()`
Design patterns	Template Method, Command
Video	CLI demo (agent) segment F06, GUI demo for the Today screen

How the classes collaborate. `ExerciseDBTool` filters the catalogue down to movements the lifter can actually perform given their equipment and limitations, so the model is choosing from a list rather than from memory. The model picks one and gives a short rationale. `ResponseParser.toSubstitution` resolves the name back to a catalogue `Exercise`, and four checks then stand between the answer and the lifter: the movement exists, the lifter can perform it, it trains what the original trained, and the load is between 40 and 110 percent of the original. Accepting applies a `SwapExerciseCommand`, which is undoable.

Error and alternative behaviour. A movement that is not in the catalogue is refused by name. No performable candidates at all produces a refusal that tells the lifter to swap manually rather than inventing something. Thirteen tests cover this feature and most of them are refusals.

F07 — Recovery Aware Session Adjustment

Field	Entry
Stage 1 reference	F07
Status	Implemented

Field	Entry
Classification	Hybrid, and the clearest division of responsibility in the project
Implemented classes	`domain/RecoveryRuleEngine`, `domain/RecoveryFlag`, `agent/agents/AdjustmentAgent`, `agent/tools/RecoveryLookupTool`, `presentation/cli/AdjustCommand`, `application/plan/ApplyAdjustmentCommand`, adjustment controls in `presentation/gui/panels/TodayPanel`
Key methods	`RecoveryRuleEngine.evaluate(RecoveryDay)`, `fallbackAdjustment(Session)`, `AdjustmentAgent.validate(Session)`, `fallback()`, `ResponseParser.toAdjustedSession(...)`, `ApplyAdjustmentCommand.apply(TrainingBlock)`
Design patterns	Template Method, Command
Video	CLI demo (agent) segment F07, GUI demo for the Today screen

How the classes collaborate. The rule engine decides *whether* to act and the agent decides *where* the reduction falls. `RecoveryRuleEngine.evaluate` returns a `RecoveryFlag` only when a reading crosses one of four thresholds. Only then may `AdjustmentAgent` run, and it must produce a session that is easier without being gutted: between 50 and 95 percent of the planned tonnage, nothing added, nothing increased.

Error and alternative behaviour. `RecoveryRuleEngine.fallbackAdjustment` is a blunt deterministic reduction used when the model is unreachable or its proposal is refused, so the lifter always trains either way. On the Today screen the Ease off button is disabled when nothing was flagged, because the decision to reduce belongs to the rule engine rather than to whoever feels like pressing a button, and `spotter adjust --fallback` shows the rule engine's own version with no model involved at all.

F08 — Max Testing and Attempt Planning

Field	Entry
Stage 1 reference	F08
Status	Implemented
Classification	Hybrid. The arithmetic needs no model; the explanation is generated and then checked against the arithmetic
Implemented classes	`domain/AttemptCalculator`, `domain/MeetAttempts`, `domain/TestDayPlan`, `agent/agents/AttemptRationaleAgent`, `PlanValidator.validateRationale`, `presentation/cli/AttemptsCommand`, `TestDayCommand`
Key methods	`AttemptCalculator.calculateMeetAttempts(Lift, double)`, `projectTestDay(Lift, double)`, `roundToPlate(double)`, `PlanValidator.validateRationale(String, MeetAttempts)`, `ResponseParser.toRationale(...)`
Design patterns	Template Method
Video	CLI demo (deterministic) segment F08 for the calculation, CLI demo (agent) segment F08 for the explanation

How the classes collaborate. `AttemptCalculator` produces an opener at 91 percent, a second at 96 and a third at 101 of the estimated maximum, each rounded to the nearest 2.5 kg plate. That is the whole feature as far as the numbers go, and `spotter attempts squat` prints them with no key configured. With `--explain`, `AttemptRationaleAgent` writes a sentence about them, and `PlanValidator.validateRationale` requires every figure appearing in that sentence to be one the calculator produced, within a tolerance of five percent.

This is the narrowest and clearest division in the project: the model is allowed to write prose about numbers it is not allowed to change.

`spotter testday` plans a heavy single with warm ups for the lifter with no meet date, which is most of them. The arithmetic had existed since the attempt calculator landed and only the way in was missing.

F09 — Macro Targets and Adherence Tracking

Field	Entry
Stage 1 reference	F09
Status	Implemented, GUI only
Classification	Deterministic
Implemented classes	`domain/MacroTracker`, `domain/MacroTarget`, `domain/MacroTotals`, `domain/DailyIntake`, `domain/FoodEntry`, `domain/FoodItem`, `infrastructure/persistence/NutritionRepository`, `SqliteNutritionRepository`, `presentation/gui/panels/NutritionPanel`
Key methods	`MacroTracker.recordIntake(LocalDate, FoodEntry)`, `consumedOn(LocalDate)`, `getRemaining(LocalDate)`, `weeklyAdherence(LocalDate)`, `NutritionRepository.log(LocalDate, FoodEntry)`, `targetFor(int)`, `saveTarget(int, MacroTarget)`, `MacroTotals.minus(MacroTotals)`
Design patterns	Repository
Video	GUI demo, segment F09

How the classes collaborate. `NutritionPanel` saves a `MacroTarget` against the lifter's profile and logs each `FoodEntry` into the day's `DailyIntake`. `MacroTracker` computes what has been consumed and what remains by subtracting totals, and `weeklyAdherence` averages the days that were actually logged rather than treating an unlogged day as a zero, which would punish the lifter for not typing rather than for not eating.

Error and alternative behaviour. `SqliteNutritionRepositoryTest` asserts that the database refuses a macro target saved against a profile that does not exist. That test exists because two earlier tests were written expecting the opposite and were wrong; the foreign key was right.

Known limitation. There is no command line route to this feature. Logging food from a terminal is a worse experience than the panel and would have been built for completeness rather than use.

F10 — Meal Suggestion

Field	Entry
Stage 1 reference	F10
Status	Implemented
Classification	Hybrid. The model chooses foods and portions; the nutrition is computed from the catalogue afterwards

Field	Entry
Implemented classes	`agent/agents/MealSuggestionAgent`, `domain/MealSuggestion` with its nested `Portion`, `infrastructure/catalog/FoodCatalog`, `SqliteFoodCatalog`, `infrastructure/sample/SampleFoods`, `agent/tools/FoodDBTool`, `PlanValidator.validateMeal`, `presentation/cli/MealCommand`, meal controls in `NutritionPanel`
Key methods	`ResponseParser.toMeal(JsonNode)`, `PlanValidator.validateMeal(MealSuggestion, DietaryConstraints, MacroTotals)`, `FoodCatalog.permittedBy(DietaryConstraints)`, `MealSuggestion.totals()`, `permittedBy(DietaryConstraints)`, `firstNotPermitted(DietaryConstraints)`
Design patterns	Template Method
Video	CLI demo (agent) segment F10, GUI demo for the Nutrition screen

How the classes collaborate. `FoodDBTool` offers the model only foods the lifter's dietary constraints permit. The model returns a list of foods and portion sizes. Crucially, `MealSuggestion` does **not** store the totals the model claimed: `totals()` recomputes them from the catalogue entries and the portion weights, so a stated figure cannot drift from the foods it describes. `PlanValidator.validateMeal` then checks the portions are plausible, between 10 and 600 grams, that there are no more than six foods, and that the meal is a sensible share of what remains for the day.

Error and alternative behaviour. A meal containing a food the lifter does not eat is refused, and `firstNotPermitted` names which one, so the refusal is actionable rather than a blank no.

F11 — Coaching Chat with Memory

Field	Entry
Stage 1 reference	F11
Status	Implemented
Classification	Hybrid
Implemented classes	`agent/agents/ChatAgent`, `agent/MemoryManager`, `agent/ConversationHistory` with its nested `Turn` and `Speaker`, `domain/MemoryNote`, `domain/ChatReply`, `infrastructure/persistence/MemoryRepository`, `SqliteMemoryRepository`, `agent/tools/PlanLookupTool`, `AnalyticsTool`, `RecoveryLookupTool`, `AdherenceTool`, `presentation/gui/panels/CoachPanel`
Key methods	`MemoryManager.remember(String)`, `wouldRefuse(String)`, `recall(int)`, `forgetEverything()`, `ConversationHistory.said(String)`, `replied(String)`, `transcript()`, `ResponseParser.toChatReply(JsonNode)`
Design patterns	Template Method, Repository
Video	GUI demo, segment F11

How the classes collaborate. `ChatAgent` answers with four lookups available to it, so a question about the current plan, recent training, recovery or adherence is answered from the database rather than from the

model's impression of the conversation. `ConversationHistory` holds the current exchange in memory, capped at twenty turns. `MemoryManager` persists only facts the lifter states, one per `MemoryNote`, capped at forty.

The gate. `MemoryManager` refuses a note written in the coach's own voice, so the coach cannot file its own suggestion as though the lifter had said it. The test for this is crude on purpose: the cost of refusing a borderline note is that the lifter repeats themselves once, and the cost of keeping a bad one is a coach that believes its own suggestions were instructions.

Eleven of the twelve pieces this feature needs were built for other features, which is the strongest evidence available that the architecture was right.

F12 — Plan Adherence and Progression Review

Field	Entry
Stage 1 reference	F12
Status	Implemented, with the adherence model simplified
Classification	Hybrid. Adherence is measured; the revision is generated and then gated
Implemented classes	`domain/PlanAdherence` with its nested `SessionOutcome`, `agent/tools/AdherenceTool`, `agent/agents/ProgressionAgent`, `PlanValidator.validateProgression`, `application/plan/ApplyProgressionCommand`, `PlanEditHistory`, `presentation/cli/ReviewCommand`
Key methods	`PlanAdherence.of(TrainingBlock, List, LocalDate)`, `tonnageRate()`, `sessionRate()`, `liftsShortOfPlan()`, `PlanValidator.validateProgression(TrainingBlock, TrainingBlock, PlanAdherence)`, `ApplyProgressionCommand.apply(TrainingBlock)`
Design patterns	Command, Template Method
Video	CLI demo (agent), segment F12

How the classes collaborate. `PlanAdherence.of` compares the block against what was actually logged and produces session outcomes, tonnage rates and the list of lifts that fell short. `spotter review` prints that with no model involved. With `--revise`, `ProgressionAgent` rewrites the remaining weeks with `AdherenceTool` available to it, and `validateProgression` gates the result. With `--keep`, the revision is applied through `ApplyProgressionCommand`.

The gate, which is the tightest in the project, because this is the only agent that rewrites training the lifter is part way through. A revision may not drop a session or a competition lift, may not add load to a lift that fell short of its plan, and may not raise anything by more than five percent in a week. The middle rule carries the feature: a lift falling behind is rarely a sign that it was too easy.

Error and alternative behaviour. `spotter review` with no active block refuses with a reason. With no API key, the adherence report still prints and says so, because it was measured rather than generated.

Simplified from the design. Stage 1 called for a verdict per prescription such as `COMPLETED_BELOW_TARGET_RPE`. Section 4, change 12.

F13 — Guided Onboarding

Field	Entry
Stage 1 reference	F13
Status	Implemented, built differently from the design
Classification	Hybrid
Implemented classes	`agent/agents/OnboardingAgent`, `domain/ProfileDraft`, `agent/ProfileValidator`, `domain/StartingStrength`, `presentation/support/PreparedProfile`, `presentation/gui/panels/ProfilePanel`
Key methods	`OnboardingAgent.parse(String)`, `ProfileDraft.isComplete()`, `missingRequired()`, `toProfile()`, `ProfileValidator.validate(ProfileDraft)`, `onlyNeedsMoreAnswers(ProfileDraft)`, `StartingStrength.needsCalibration(List, Lift)`
Design patterns	Template Method, Builder
Video	GUI demo, segment F13

How the classes collaborate. The lifter writes a paragraph about themselves. `OnboardingAgent` reads it into a `ProfileDraft`, which is an immutable fluent builder rather than a profile, so an unanswered question stays unanswered instead of being filled with a default. `ProfileValidator` checks the draft, and `onlyNeedsMoreAnswers` distinguishes a draft that is wrong from one that is merely incomplete. `ProfilePanel` shows what was understood alongside what is still missing, and nothing is saved until the lifter confirms.

Why a draft rather than a profile. A `LifterProfile` fills in defaults, so parsing straight into one would turn a question the lifter never answered into an invented fact. Section 4, change 2.

What happens next. `StartingStrength` runs once a profile is saved, so onboarding ends with something to do rather than an empty application. It distinguishes a lifter with nothing logged, who must supply something before anything can be planned, from one with a little history, who only has to keep going.

Changed from the design. Stage 1 specified an adaptive eight question interview. Section 4, change 18.

F14 — Adopt an Existing Programme

Field	Entry
Stage 1 reference	F14
Status	Implemented, all three routes
Classification	One hybrid route and two fully deterministic routes
Implemented classes	`infrastructure/csv/ProgrammeImportService`, `agent/agents/ProgrammeParserAgent`, `PlanValidator.validateAdoptedBlock`, `ResponseParser.toBlock`, `domain/BlockOrigin`, `presentation/cli/AdoptCommand`

Field	Entry
Key methods	`ProgrammeImportService.fromManualEntry(List, LocalDate)`, `fromCsv(Path, ColumnMapping, LocalDate)`, `problems()`, `PlanValidator.validateAdoptedBlock(TrainingBlock, LifterProfile, Map)`
Design patterns	Template Method, Adapter, Factory Method
Video	CLI demo (agent), segment F14

How the classes collaborate. There are three ways in and only one involves a model. A programme typed line by line, as `1/2 Back Squat 5x5@140 rpe8`, is parsed by a regular expression in `fromManualEntry`. A programme exported as a spreadsheet is read by `fromCsv` reusing `ColumnMapping` from F02 pointed at different fields. A programme pasted as prose goes to `ProgrammeParserAgent`. All three then pass through `validateAdoptedBlock`, so a programme is held to the same standard however it arrived, and all three produce a block marked `BlockOrigin.ADOPTED`.

Why the gate is a different gate. An adopted programme was written by somebody else, so holding it to the lifter's own progression would reject every programme that does not match the scheme Spotter would have chosen, which is most of them. `validateAdoptedBlock` checks possibility instead: the movements exist and can be performed, the week is not busier than the lifter said they train, and no load is one nobody could lift. A load far above a current maximum nearly always means the text was misread rather than that the coach is bold.

Why two routes are deterministic. A spreadsheet or a typed line already has a structure. Sending either to a model would be paying for reading comprehension that is not required and making a deterministic import depend on a network call.

Error and alternative behaviour. A line that does not parse is refused with the line quoted back. Rows passed over in a spreadsheet are listed rather than silently dropped. This is also the only feature with no prepared reply for running without an API key, and the command says so outright rather than pretending.

What this feature cost to build. Stage 1 called it the hardest feature in the design and the Stage 2 plan named it the first to drop. The free text route turned out to be the cheap one, because `CoachAgent` already supplied the retry, `ResponseParser` already resolved exercise names and built weeks from JSON, `BlockRepository` already stored a block and `BlockOrigin.ADOPTED` already existed. What had to be written was a prompt, a gate and a command. That saving is the whole argument for Template Method, arriving two months after the decision to use it.

New features

The four features below were not in Stage 1. They are listed here so that every implemented feature is accounted for, mapped and demonstrated. Section 4 gives the original need, the reason and the impact of each.

N01 — Training Report Export

Field	Entry
Stage 1 reference	New

Field	Entry
Status	Implemented
Classification	Deterministic
Implemented classes	`application/report/ReportBuilder`, `application/report/TrainingReport` with its nested `Section`, `infrastructure/report/ReportWriter`, `PdfReportWriter`, `CsvReportWriter`, `presentation/gui/panels/ReportPanel`
Key methods	`ReportBuilder.build(LocalDate)`, `ReportWriter.write(TrainingReport, Path)`, `extension()`, `ReportPanel.export(ReportWriter)`
Design patterns	Strategy: `ReportWriter` with one implementation per format
Video	GUI demo, segment Reports

How the classes collaborate. The lifter chooses PDF or CSV on the Reports screen. `ReportPanel` calls `ReportBuilder.build`, which reads the profile, analytics, current block, recovery and nutrition through the same repositories the other screens use and returns one `TrainingReport` made of sections. `ReportPanel.export` passes that report to the chosen `ReportWriter`, which formats it and writes the file. Adding a third format means adding a writer; the builder and the screen do not change. No model is involved, so it works with no API key.

N02 — Ask Coach Drawer

Field	Entry
Stage 1 reference	New surface for F11
Status	Implemented
Classification	AI based, the same agent as F11
Implemented classes	`presentation/gui/DashboardApp` with `withDrawer` and `setDrawer`, `presentation/gui/panels/CoachPanel`, `agent/agents/ChatAgent`
Key methods	`DashboardApp.setDrawer(boolean)`, `CoachController` chat calls through the Facade
Design patterns	Facade, as for F11
Video	GUI demo, segment Ask coach

How the classes collaborate. The floating button calls `setDrawer(true)`, which slides the same `CoachPanel` over whichever screen is open. The panel sends the question to `ChatAgent` through `CoachController` exactly as the Coach tab does, so the coach has the same memory and the same tools. The lifter does not lose their place on the current screen.

N03 — Light and Dark Themes

Field	Entry
Stage 1 reference	New
Status	Implemented
Classification	Deterministic
Implemented classes	`presentation/gui/ThemePreference`, the `.root.light` rules in `src/main/resources/gui/spotter.css`

Field	Entry
Key methods	`ThemePreference.toggle(Parent)`, `apply(Parent)`, `isLight()`
Design patterns	N/A
Video	GUI demo, segment Light and dark

How the classes collaborate. The icon at the top right calls `ThemePreference.toggle` on the root node. The class stores the choice and adds or removes the `light` style class, and the stylesheet restyles every screen from that one class, so no panel knows which theme is active.

N04 — Guided Tour

Field	Entry
Stage 1 reference	New
Status	Implemented
Classification	Deterministic
Implemented classes	`presentation/gui/GuidedTour` with its nested `Step`, `DashboardApp.buildTour`
Key methods	`GuidedTour.start()`, `stop()`, `next()`, `Step.seconds()`
Design patterns	N/A
Video	GUI demo, whole recording

How the classes collaborate. The ? button, or `SPOTTER_TOUR=1`, calls `GuidedTour.start`. `DashboardApp.buildTour` supplies fourteen steps, each a caption and an action that opens a screen. `GuidedTour` runs the action, shows the caption, and advances when a timer sized to the caption length finishes. It only looks: it never saves, imports or calls the model, so running it changes no data.

Supporting work: machine readable output

Not a Stage 1 feature, and listed here for completeness rather than counted as one. The eight commands that answer a question take `--json` and print a single object with the same four keys: `command`, `ok`, `value` and `trace`. A refusal prints `ok` false with a reason and keeps the exit code the prose path used. Implemented in `presentation/cli/JsonOutput` and covered by fifteen tests in `JsonOutputTest`, which run all eight commands against a temporary database. This exists for the Stage 3 KUMA harness.

---

3. Implemented design patterns

Stage 1 named eight patterns. All eight are implemented, against a course minimum of five. Builder and Repository are also present as supporting patterns.

3.1 Template Method

Design problem. Nine agents all need the same sequence: gather context, offer tools, call the model, parse the reply, validate it, and on refusal send the reason back and try again. Written nine times, that sequence would drift, and the one part that must never be skipped is the validation.

Participating classes. `agent/CoachAgent` is the abstract class. Its `run()` is the template method and is declared `final`. The primitive operations are `systemPrompt()`, `gatherContext()`, `describeTask()`, `createToolset()`, `parse(String)` and `validate(T)`. The nine concrete classes are `BlockGenerationAgent`, `SubstitutionAgent`, `AdjustmentAgent`, `AttemptRationaleAgent`, `MealSuggestionAgent`, `OnboardingAgent`, `ChatAgent`, `ProgressionAgent` and `ProgrammeParserAgent`.

Why it is useful here. `run()` being final is the design decision, not an accident of syntax. It means no agent can reimplement the retry or skip the validation, which is the guarantee the whole project depends on. The payoff was measurable: F14, named in Stage 1 as the hardest feature, needed only a prompt, a gate and a command because the loop already existed.

Matches Stage 1. Yes, with `MAX_TOOL_ROUNDS` raised from 4 to 6 and `MAX_RETRIES` from 1 to 2. Section 4, change 4.

3.2 Strategy

Design problem. Two separate axes of variation. How a load is chosen depends on the lifter's goal and experience. How the model is reached depends on whether this is production, a test, or a replay.

Participating classes. `domain/ProgressionStrategy` is the strategy interface, with `AbstractProgression` holding the shared behaviour and `RPEProgression`, `LinearProgression`, `NoviceLinearProgression` and `PercentageProgression` as concrete strategies, selected by `ProgressionStrategyFactory.forGoal`. Separately, `agent/LLMClient` is a strategy interface with `ClaudeClient` and `ScriptedLLMClient` as implementations.

Why it is useful here. The second use is what makes the project testable. All 502 tests run against `ScriptedLLMClient` with no network and no API key, because nothing above the interface knows which implementation it has.

Matches Stage 1. Yes, with `AbstractProgression` added between the interface and the four implementations to hold shared behaviour.

3.3 Facade

Design problem. Eight screens and fifteen commands all need repositories, catalogues and the deterministic services. Without a single front door, each one constructs its own and they disagree about which profile is current.

Participating classes. `application/CoachController`.

Why it is useful here. Nothing else in the application opens a database connection or constructs a repository.

Differs from Stage 1. The Stage 1 facade had about twenty five methods including one per agent. The implemented facade covers the repositories and the deterministic services and does not wrap the agents. Section 4, change 17.

3.4 Adapter

Design problem. Hevy, Whoop and arbitrary applications all export CSV, and no two agree on column names, date formats or units.

Participating classes. `infrastructure/csv/DataSource<T>` is the target interface. `HevyCsvAdapter`, `WhoopCsvAdapter` and `GenericCsvAdapter` are the adapters, and `CsvFileReader` is the adaptee they share.

Why it is useful here. `DataSource` is generic in what it produces, so a workout adapter and a recovery adapter are the same interface over different payloads. `read()` returns the rows it could parse and `skipped()` the ones it could not, rather than throwing, because one unreadable row in a real export should not lose the other four hundred.

Matches Stage 1. Yes.

3.5 Factory Method

Design problem. Three places need to choose an implementation at runtime from information only available then.

Participating classes. `DataSourceFactory.create` and `createMapped` choose an adapter from a detected `SourceKind` or a supplied mapping. `ProgressionStrategyFactory.forGoal` chooses a progression from a goal and an experience level. `CoachAgent.createToolset()` is a factory method on the template, where each agent decides which tools the model may call.

Why it is useful here. The third is the interesting one, because it is a factory method in the strict sense: the superclass defines when the toolset is built and the subclass decides what it contains.

Matches Stage 1. Yes.

3.6 Decorator

Design problem. Caching and recording are both useful around an LLM client and neither belongs inside one.

Participating classes. `LLMClientDecorator` is the abstract decorator implementing `LLMClient` and holding a wrapped `LLMClient`. `CachingLLMClient` returns a stored reply for an identical request. `RecordingLLMClient` writes replies to disk for replay.

Why it is useful here. Either can wrap either, and the client being wrapped cannot tell. `CachingLLMClient` matters most when the Stage 3 suites rerun the same scenarios.

Matches Stage 1. Yes.

3.7 Observer

Design problem. Saving a plan on one screen changes what three other screens show, and none of them should have to know about the others.

Participating classes. `application/event/EventBus` is the subject, `EventListener` the observer interface, `SpotterEvent` the event type, and `Panel.changedBy()` is how each screen declares what concerns it. `DashboardApp` subscribes the panels and acts on the notifications.

Why it is useful here. It changed real behaviour rather than adding structure beside it. Panels used to refresh on every visit, which was correct and wasteful. Now a panel that saves something publishes and does not refresh itself; the shell refreshes the panel on screen at once and marks the others stale, so they redraw when next opened and not before.

Three events exist and only three, because those are the changes the application makes to itself while running. Logging a set and importing recovery readings change a screen too, but both arrive from a separate process that a bus in this window cannot hear, and inventing events with no publisher would be pattern for its own sake. A panel that names no event is refreshed on every visit as before, which is what Analytics does.

One rule is worth naming: a listener that throws does not stop the rest from being told, and the failure is kept rather than swallowed. A panel failing to redraw is a bad panel; it should not also become the reason every other screen quietly stops updating.

Differs from Stage 1. Stage 1 had a `BasePanel` abstract superclass. `Panel` stayed an interface with a default method. Section 4, change 15.

3.8 Command

Design problem. A lifter eases off a session at seven in the morning after a bad night, feels fine by the evening, and should be able to put the session back rather than regenerate the plan.

Participating classes. `application/plan/PlanEditCommand` is the command interface, with `describe()`, `apply(TrainingBlock)` and `undo(TrainingBlock)`. `ReplaceSessionCommand` is an abstract base for the two commands that replace a single session, `SwapExerciseCommand` and `ApplyAdjustmentCommand`; `ApplyProgressionCommand` implements the interface directly because it replaces the whole block. `PlanEditHistory` is the invoker and holds the stack. `TodayPanel` is the client.

Why it is useful here. A command returns a new block rather than mutating the one it was given, which makes undo a matter of keeping the old value rather than of reversing arithmetic correctly. The history is bounded at twenty edits, because an undo stack that grows all day is a memory leak with a friendly name.

Matches Stage 1, with `ChangeLoadCommand` replaced by the shared `ReplaceSessionCommand` base. The stack lives in the window rather than the database. Section 4, change 14.

3.9 Supporting patterns

Builder in `PromptBuilder`, `LifterProfile.Builder` and `RecoveryDay.Builder`, each for an object with many optional parts that should not exist half constructed. **Repository** in `Repository<T>` and its implementations, which is what keeps SQL out of the domain entirely.

3.10 Updated UML class diagram

The class diagram was regenerated from the implemented code on 4 November 2026. It is one model, `docs/diagrams/sources/model.iuml`, filtered into five views so the views cannot contradict each other, and delivered as both UMLet `.uxf` files and PNG exports.

View	Classes	File
Complete model	144	`docs/diagrams/png/class-full.png`
Presentation and application	26	`docs/diagrams/png/class-presentation-application.png`
Domain	53	`docs/diagrams/png/class-domain.png`
Agent	41	`docs/diagrams/png/class-agent.png`
Infrastructure	24	`docs/diagrams/png/class-infrastructure.png`

The Stage 1 use case and sequence diagrams remain accurate and were not regenerated. The class diagram was, because the implementation had moved: see section 4 for what changed and why.

The figures were exported on 4 November. Presentation classes added afterwards (`ImportPanel`, `ProfileEditor`, `ReportPanel`, `GuidedTour`, `ThemePreference`) and the report classes behind the Reports screen (`ReportBuilder`, `TrainingReport`, `ReportWriter`, `CsvReportWriter`, `PdfReportWriter`) are not drawn. They extend the existing structure and add no new relationships between layers, so the figures remain correct for everything else.

4. Changes and rationale

Twenty five changes are recorded. Each was written at the time the decision was made and names the commit that carried it, so the claim can be checked against the code rather than taken on trust. The full text of each lives in `docs/stage2/design-changes.md`.

Changes fall into three kinds. Some are design decisions that only became visible once the code existed. Some are corrections forced by real data, which Stage 1 could not have anticipated because Stage 1 had no data. The rest are constraints imposed from outside, such as a library or a model that changed under the project.

4.1 Feature level changes

Change 18 — F13 Guided Onboarding: an interview became a paragraph

1. **Original Stage 1 feature.** F13, Guided Onboarding. An `OnboardingAgent` asking at most eight questions one at a time, adapting each question to what had already been answered.
2. **Implemented feature and behaviour.** The lifter writes one paragraph about themselves. The agent reads it into a `ProfileDraft` and the Profile screen shows what was understood alongside what the lifter has not said. Nothing is saved until the lifter confirms.
3. **What changed and why.** The interview is the expensive version of the same thing. Eight adaptive questions is eight model calls and eight screens. A paragraph is one call and one screen, and a lifter who leaves something out is asked for exactly that, which is what the adaptive part of the interview existed to achieve.
4. **How it addresses the original need.** The original need was a lifter reaching a usable profile without filling in a form they do not understand. A paragraph in their own words does that in less time. The unanswered questions are listed rather than asked one at a time, so nothing is guessed and the flow is shorter.
5. **Affected use cases, classes, methods and patterns.** UC covering onboarding. `OnboardingAgent`, `ProfileDraft`, `ProfileValidator`, `ProfilePanel`. `ProfileDraft.isComplete()`, `missingRequired()`, `ProfileValidator.onlyNeedsMoreAnswers()`. Template Method and Builder unaffected.

Change 12 — F12 Plan Adherence: per prescription verdicts became per session tonnage

1. **Original Stage 1 feature.** F12, with `PlanAdherence.compare()` assigning a verdict per prescription such as `COMPLETED_BELOW_TARGET_RPE` or `MISSED_REPS`, carried in `AdherenceReport`, `AdherenceEntry` and `AdherenceVerdict`.
2. **Implemented feature and behaviour.** `PlanAdherence.of()` compares planned against logged tonnage per session, plus the heaviest load reached against the heaviest prescribed per lift, and reports which lifts fell short.
3. **What changed and why.** Writing the progression agent showed that a per prescription verdict was precision nobody used. The three classes carrying it collapsed into one.
4. **How it addresses the original need.** The need was to separate a block that is not working from one that was not done, which is what the progression decision turns on. Tonnage per session answers

that, and needs no judgement about what a near miss on a single set means.
5. **Affected classes and patterns.** `PlanAdherence` with a nested `SessionOutcome` record, replacing `AdherenceReport`, `AdherenceEntry` and `AdherenceVerdict`. `AdherenceTool` and `validateProgression` read the simpler shape. No pattern affected.

Smaller feature level changes

F08 gained `spotter testday`. The attempt arithmetic had existed since `AttemptCalculator` landed and only the way in was missing, so a feature useful to every lifter was reachable only by someone preparing to compete. Stage 1 called the command `spotter test plan`; it is one word rather than two.

F02 gained `spotter log --as-prescribed`. Typing out a session you have just completed exactly as it was written down for you is the kind of work that makes people stop logging.

F03 gained `spotter checkin`. Without it the recovery feature was conditional on owning a particular wearable.

None of these replaces anything. Each makes an implemented capability reachable.

Smaller fixes found while recording the demonstrations: the Today screen shows the next session when today is a rest day, the Recovery screen says when the latest reading was when the chosen weeks hold none, and every screen opens scrolled to the top.

4.2 Pattern level changes

Observer: `BasePanel` became an interface default method

6. **Original pattern.** Observer, with `BasePanel` as an abstract superclass whose subclasses listened on an `EventBus`.
7. **Pattern implemented.** Observer, with `EventBus` as subject, `EventListener` as the observer interface, and `Panel.changedBy()` as a default interface method naming the events that concern each screen.
8. **Participating classes and roles.** `EventBus` subject, `EventListener` observer, `SpotterEvent` the event type, `Panel` implementors as concrete observers, `DashboardApp` as the client that subscribes them.
9. **Design problem addressed.** Saving a plan on one screen changes what other screens show, and none of them should have to know about the others.
10. **Why appropriate, and the effect on the architecture.** An interface with a default method gives the same subscription without forcing every screen to inherit from a shared parent. The effect on the architecture is that a screen's only obligation is to name what it displays; the shell does the subscribing, so a new screen cannot silently forget to register itself.

Command: `ChangeLoadCommand` became a shared `ReplaceSessionCommand` base

6. **Original pattern.** Command, with `SwapExerciseCommand`, `ChangeLoadCommand`, `ApplyAdjustmentCommand` and `ApplyProgressionCommand` each implementing the interface directly.

7. **Pattern implemented.** The same, except that swapping an exercise and easing a session off both come down to replacing one session, so they share an abstract `ReplaceSessionCommand`, and `ChangeLoadCommand` was not needed.
8. **Participating classes and roles.** `PlanEditCommand` command interface, `ReplaceSessionCommand` abstract command, `SwapExerciseCommand` and `ApplyAdjustmentCommand` concrete commands, `ApplyProgressionCommand` concrete command implementing directly, `PlanEditHistory` invoker, `TodayPanel` client, `TrainingBlock` receiver.
9. **Design problem addressed.** An edit should be reversible without the code that applies it and the code that reverses it being able to drift apart.
10. **Why appropriate, and the effect on the architecture.** Writing apply and undo together in one class is the point of the pattern. The architectural effect is that `TodayPanel` no longer rebuilds blocks itself; it hands the history a change that knows how to reverse itself.

Facade: `CoachController` narrowed

6. **Original pattern.** Facade, with about twenty five methods including one per agent.
7. **Pattern implemented.** Facade over the repositories, catalogues and deterministic services only.
8. **Participating classes and roles.** `CoachController` facade, over `Database`, the five repositories, the two catalogues, `CsvImportService`, `StrengthAnalytics`, `AttemptCalculator` and `RecoveryRuleEngine`.
9. **Design problem addressed.** Eight screens and fifteen commands each constructing their own repositories would disagree about which profile is current.
10. **Why appropriate, and the effect on the architecture.** The agents do not have a common shape: `BlockGenerationAgent` takes a profile, a catalogue, a history, a strategy, a date and a week count, while `AttemptRationaleAgent` takes a set of attempts and an estimated maximum. A method per agent would be a second interface carrying the same parameters, kept in step by hand, and every new agent would mean editing the facade as well. The cost is that the presentation layer knows the agent constructors; `GuiContext` and `ModelAccess` exist so that cost is paid once rather than in every panel.

4.3 Consolidated change record

#	Original design	Implemented change	Reason	Impact
1	One `PlanValidator` gates every agent	Added `ProfileValidator`	Judging loads, meals, explanations and profiles in one class is four responsibilities sharing a name	Two small validators with one reason to change each
2	Agent produces a `LifterProfile`	Agent produces a `ProfileDraft`	A profile fills in defaults, so an unanswered question becomes an invented fact	Onboarding can report what it does not know
3	Exercise names resolved at parse time only	Loads also checked through `LoadRangeTool` before the model answers	Rejection teaches one attempt at a time; a lookup means it never has to be told	Fewer refusals and fewer retries, so lower cost

#	Original design	Implemented change	Reason	Impact
4	`MAX_TOOL_ROUNDS` 4, `MAX_RETRIES` 1	6 and 2	Both limits were chosen against a test catalogue of two exercises	A four week block fits inside the loop
5	Model client built once per tier	Built per tier and reply length	A cached model fixed the reply length for every agent after the first	Long replies stopped being truncated
6	`Range.contains` exact	Allows 0.05 kg	A bound of 66.04 printed as 66.0 and refused 66.0	A correct load is no longer refused for a rounding artefact
7	Limitations avoid movement patterns	Also avoid named exercises	The domain supported both since Stage 1; the schema did not	A limitation can name the one movement that hurts
8	Conversation persisted	Held in memory, only stated facts stored	A stored conversation stores whatever the model got wrong and reads it back as true	Memory contains things the lifter said and can correct
9	Memory notes free form	Notes refused when written in the coach's voice	A coach that remembers its own advice as the lifter's words cannot be corrected	The gate is crude on purpose; a false refusal costs one repetition
10	Model identifiers fixed in config defaults	Updated twice, now pinned with an explanation	The identifiers written in Stage 1 were retired before Stage 2 called them	Every call failed until corrected; now pinned to versions the client can parse
11	`spotter.db` beside the working directory	`--env` added to every command	Running the GUI and the CLI against different databases was too easy	The test suite can point at a temporary database
12	Validation called after the agent returns	Validation moved inside the agents	A rejected proposal must go back to the model with the reason attached, and the retry loop lives where the conversation lives	`CoachAgent.run()` could be made final
13	Competition lift checks apply to any exercise mapped to a lift	Known limitation, fix identified, not made	A close grip bench is held to competition bench loads, which is wrong	Recorded honestly; the fix touches the estimated maxima too
14	Undo stack, lifetime unspecified	Held in the window, bounded at twenty edits	The case it exists for happens inside one sitting	Undo is a button, not a regeneration; it does not survive a restart
15	Panels refresh every visit	Panels refresh when something they show has changed	Walking the navigation column re-queried the database for figures nothing had touched	Fewer queries, and the panel on screen updates immediately
16	Commands print prose	Eight commands also take `--json`	Stage 3 drives the application from a harness, and scraping prose would make every wording change a breaking change	A stable contract for Stage 3

#	Original design	Implemented change	Reason	Impact
17	`CoachController` wraps every agent	Facade covers repositories and deterministic services only	The agents take different inputs, so a method per agent would be a second interface kept in step by hand	Adding an agent no longer means editing the facade
18	F13 as an eight question adaptive interview	A single paragraph read into a draft	Eight adaptive questions is eight model calls and eight screens for the same outcome	One call, one screen, and the gaps are listed rather than asked
19	No check that a prescribed load was above zero	A movement needing equipment is refused at zero load	A real run prescribed Romanian deadlifts at 0.0 kg and the gate passed it	The claim that no model output reaches the lifter unchecked is now true for loads as well as ranges
20	Import, profile editing and a light theme existed only as commands or not at all	The window gained an Import screen, an editable profile form and a theme toggle	A lifter should not need a terminal to bring in their data	The whole workflow works from the window
21	The coach was one tab, and nothing could be exported	A quick coach drawer on every screen, and a Reports screen with PDF and CSV export	A lifter mid session should not leave the screen to ask, and a coach or a spreadsheet needs the numbers in a file	No screen has to be left to ask a question, and a report can be kept
22	A ten item sidebar put rarely used screens beside daily ones	A top bar with eight tabs; Import, Profile and the theme icon at its right	Import and Profile are done now and then, so they should not compete with the screens opened daily	The tabs are the screens opened every day
23	The application assumed a lifter would find their own way around	A guided tour with a caption on every screen	The first minute with an application and a silent recording both need the screen to explain itself	A viewer with no sound can follow every screen
24	The importer created whatever exercise name it met	The standard exercise catalogue is loaded on start, and a bare exercise is completed later	On an unseeded database every imported lift was unknown to analytics and Plan	An import works on a fresh database, and an existing one repairs itself
25	Only Plan printed tokens and cost	Every agent trace ends with tokens used and what the calls cost	The cost of an agent step is part of understanding it, and the recording should show it	Cost is visible at the end of each trace

5. AI tools and collaboration log

5.1 Tool used to build the software

Tool	LLM model	Purpose	Example tasks
Claude Code, in the Claude desktop application	Claude Opus 5 configured for the session; see the note below	Writing, reviewing and debugging the implementation, and drafting the Stage 1 and Stage 2 documents	Scaffolded the Maven project and SQLite schema; wrote the domain calculations and their tests; wrote the nine agents against the `CoachAgent` template; built the eight JavaFX panels; added the Command and Observer patterns; wrote `JsonOutput` and its tests; drafted the design changes log and the feature mapping

On the model identifier. The session is configured for `claude-opus-5`. The tool does not expose the model serving any individual request, and the runtime may substitute a different model mid session, so the configured identifier is what can be stated accurately and the serving model for a given request cannot. The Stage 2 instruction asks that this be said plainly rather than guessed at.

No other AI tool was used: no second assistant, no autocomplete model, no code generation service.

Cost. The plan used was Claude Pro, at 31.64 Canadian dollars a month, billed as a flat subscription rather than per token, so there is no token figure for this part and no portion of the fee is attributable to this project: the same subscription covered all of the author's use over the period. The cost that does vary with this project is the API spend in 5.3.

5.2 Model the software itself calls

Tier	Model identifier	Used by	Why
Ordinary	`claude-haiku-4-5-20251001`	`SubstitutionAgent`, `AdjustmentAgent`, `AttemptRationaleAgent`, `MealSuggestionAgent`, `OnboardingAgent`	These pick from a filtered list or write one checked sentence, and the answer is validated either way
Reasoning	`claude-sonnet-4-6`	`BlockGenerationAgent`, `ChatAgent`, `ProgressionAgent`, `ProgrammeParserAgent`	These write or rewrite a whole block, or interpret prose

Both identifiers are set in `.env` and documented in `.env.example`; neither is hard coded. They are pinned to the 4 series deliberately: the 5 series emit a `thinking` content block that LangChain4j 0.35.0 cannot parse, so a call to one fails before the reply reaches the parser. The fix is a client library upgrade rather than a model change.

5.3 Token and money cost

What	Calls	Input tokens	Output tokens	Cost
First live run of the agents, including every call that failed or was refused	9	18,443	12,460	~$0.24, measured by the application
2 November 2026, the first day against the real API (console)		56,326	45,807	$0.82
4 to 8 November, fixes, retesting and the demonstration recordings (console)		220,406	104,741	about $2.0 at list rates
Whole project, every live call (console, 2 to 8 November)		276,732	150,548	**about $2.9**

The rows marked console come from the account usage export, and they are the project's total because the key was never used for anything else. The export gives tokens by day and model but no call counts and no dollar amounts, so the call count is left blank and the cost of the later days is calculated at list rates: 3 and 15 dollars per million input and output tokens for Sonnet, 1 and 5 for Haiku. The first console row is 2 November 2026, the day the agents were first run against the real API. The second covers 4, 5, 6 and 8 November.

The 2 November figures check out against the published rates, which matters because the application's own cost reporting uses them. At the reasoning model's rates of 3 and 15 dollars per million, 56,326 input and 45,807 output tokens come to 86 cents against a console figure of 82. The four cent gap is the calls that went to the fast tier at 1 and 5, which is what the model tiering predicts.

The total counts the calls that failed against retired model identifiers and the proposals the validator refused. That is the honest number: a refused proposal costs what an accepted one costs, and this project refuses plenty by design.

The demonstration recordings. The agent CLI demo was made against the live model in November, and every model step prints its own cost at the end of its trace. The steps that print one came to about 20 cents: the two week block 9 cents, the recovery adjustment 3 cents, the revision of a block 5 cents, the meal suggestion 1 cent, and the rest under a cent each. These calls are part of the 4 to 8 November row above.

Why the cost is as low as it is. Most of what Spotter does involves no model. Nine of fifteen commands never call one. Where a model is involved it proposes and deterministic code disposes, and all 510 tests run against `ScriptedLLMClient` with no network.

5.4 Representative collaboration log

The full log is `docs/stage2/ai-collaboration-log.md`, nine entries covering ninety four commits, one per working session. Four representative interactions follow.

Retired model identifiers. *Task:* get the agents running against the real API. *Instruction:* configure a key and run them. *AI contribution:* suggested `claude-3-5-haiku-20241022` and `claude-3-5-sonnet-20241022` from its own training rather than checking. *Human contribution:* every call failed; I had it look up the current model list rather than guess again. Both identifiers had been retired, in February 2026 and

October 2025. *Outcome:* current identifiers configured, then pinned with the reason written into `.env.example`. Change 10.

A claim that something had been done. *Task:* replace the pinned model identifiers in `.env`. *AI contribution:* stated the edit had been made. *Human contribution:* I re-ran the command and hit the identical error, and said so. The edit had not been made. *Outcome:* the edit was made for real. The lesson is the one the instruction makes: AI output must be inspected, including the claim that something was done.

A test that was wrong about its own code. *Task:* test block generation refusals. *AI contribution:* a test asserting the failure message said "4 weeks". *Human contribution:* the message the code produced said "4 were asked for". I had the test fixed rather than the message, because the message was better. *Outcome:* `BlockGenerationAgentTest` asserts the real wording. The same session found two repository tests saving a macro target against a profile that did not exist and passing because the test expected that; the foreign key was right, and a third test now asserts the refusal.

A correction that went the other way. *Task:* design the chat coach's memory. *Human contribution:* I had assumed it would persist the conversation, which is what a chat feature normally does. *AI contribution:* argued for storing only facts the lifter states, behind a gate, because storing a conversation stores whatever the model got wrong and reads it back later as true. *Outcome:* I accepted it. Changes 8 and 9. Recorded here because a log that credits the human for every good decision is not worth reading.

---

6. Development and verification summary

6.1 Weekly commit summary

One hundred and twenty one commits in total, of which eighty seven fall inside the three Stage 2 reporting periods. Every period exceeds the minimum of five meaningful code commits.

Period	Dates	Commits	Of which code
1	20 to 26 October 2026	7	6
2	27 October to 2 November 2026	41	40
3	3 to 9 November 2026	39	22

Base URL for all links: `https://github.com/devparikh28/EECS-3311-Project--Spotter/commit/`

Period 1, the skeleton: Maven build and local key configuration `1b2cf81`, configuration loading with safe failure `162a0be`, JUnit tests for configuration `2c9ac7e`, SQLite schema and the repository interface `1260fd1`, picocli interface and the status command `8705805`, domain enums and value objects `e583b5b`, core training entities `16db05b`.

Period 2, the domain, the infrastructure and the agents: RPE chart and strength analytics `0c70311`, attempt and test day planning `8ab4e80`, recovery evaluation and the deterministic fallback `95c997f`, reading real Hevy exports and fixing what real data exposed `dd73edb`, the agent core `3e15fdf`, the Claude client and the decorators `18000a4`, the first real coaching agent `8efc6b0`, the dashboard shell `02a5f49`, the chat agent `86d43c6`, plan adherence `2c31026`, and the five fixes that came out of the first live run, `3ebec0f`, `2e8ae26`, `7f4bcdd`, `069d842`, `e641653`.

Period 3, the remaining features and the patterns: closing the three gaps the Stage 1 check found `604076e`, adopting an existing programme `4957fa0`, reviewing and revising a block `0c78fc0`, reading anybody's export `6f6b421`, the typed and spreadsheet routes for F14 `8b18e6e`, the Command pattern `aceab2f`, the Observer pattern `5b35c28`, and the JSON output `d8fe005`. From 5 to 8 November the interface work: import, profile editing and the light theme `8af1dea`, the Ask coach drawer and reports `8e1f7e2`, the top bar `325e954` and `543d249`, a separate worker for chat `491606b`, the guided tour `567d097`, and the fixes found while recording the demonstrations: Today on a rest day `03f4bbb`, screens opening at the top `f8abda0`, the recovery empty state `a873a17`, the exercise catalogue loaded on start `04663ff`, and the cost line on every trace `4f0d68a`.

6.2 Checks performed

Check	Result
Automated test suite	510 tests, all passing, no network and no API key required
Live end to end run against the real API	2 November 2026, nine calls, block generation refused then corrected and accepted
GUI verified headless	Every panel navigated twice under Xvfb with no errors
CLI quick start	Every command in the README run from a clean database before being documented

Check	Result
JSON output	All eight commands parsed as JSON by an automated test, which catches any stray print statement in the prose path
Class diagram against code	Every class in the model checked against the source; no arrow references an undeclared class
Demonstration recordings	Three recordings made and each reviewed frame by frame for the wrong data, a clipped screen or an unreadable caption; two were redone because of what the review found
Secrets	No API key appears in any source file, report, log or committed output; `.env` is gitignored and `.env.example` holds placeholders only

6.3 Important defects found and corrected

Defect	How it was found	Fix
Hevy adapter broke on a real export	Importing an actual 7318 row file	Four wrong assumptions corrected, `dd73edb`
Re importing the same file crashed	Importing the same export twice	Duplicate detection, `f4f6e0e`
Agent returning an empty reply after exhausting its tool rounds, reported as the model's fault	Running block generation for real	One further call with no tools offered, `7f4bcdd`
Long replies silently truncated	A four week block came back cut off	Model cached per name and reply length, not name alone, `e641653`
A correct load of 66.0 kg refused against a range printing as 66.0	Reading a refusal that looked wrong	Boundary tolerance in `Range`, change 6
Two repository tests passing while saving an orphaned macro target	Reviewing AI written tests	Tests corrected and a third added asserting the refusal
Importing into an empty database left every exercise unknown, so analytics and Plan reported no training history	Recording the demonstration from a clean database	The standard exercise catalogue is loaded on start and a bare exercise is completed when the real one arrives, `04663ff`. Change 24
A two week block prescribing Romanian deadlifts at 0.0 kg	Watching a real run while recording the demonstration videos	A movement requiring equipment is refused at zero load, with the equipment named; bodyweight movements still pass at zero. Change 19

The first live session found six problems in under two hours, three in the project's own code and three in assumptions about the model. A seventh and an eighth were found later, in the same way, by running the application for an audience while recording the demonstrations. Every one of the eight came from running the thing rather than from writing another test, which is the main lesson of the implementation period.

6.4 Remaining limitations

The competition lift variation defect, change 13. F09 having no command line route. F14 having no offline demonstration mode. The undo stack not surviving a restart. Stage 3 not started: `tools/kuma/` does not exist yet, though the `--json` output it will drive does.

---

7. Demonstration index

Every implemented feature, including the four new ones, has a demonstration. The recordings are grouped into three, each with clearly labelled segments; every feature below gives the recording and the timestamp of its segment.

CLI demo, deterministic commands (https://youtu.be/IFqLJAf8m-g). Shows that the figures a lifter trains off are calculated rather than generated, and work with no API key configured.

CLI demo, the agent (https://youtu.be/R0Z8aMzREhE). Shows the tool calls, the validation refusals and the retries, with `--trace`, and the tokens and cost at the end of each trace.

GUI demo, the graphical interface (https://youtu.be/QMOtI4h0Y5E). A captioned guided tour of every screen: Dashboard, Today, Plan, Analytics, Recovery, Nutrition, Coach, the Ask coach drawer, Reports, Import and Profile, and the light and dark themes. The tour only looks and changes no data.

Feature	GUI demo	CLI demo
F01 Lifter Profile and Constraints	2:05	
F02 Workout Logging and Import		0:13
F03 Recovery Input		1:46
F04 Strength Analytics	0:50	2:41
F05 Training Block Generation	0:38	0:19
F06 Equipment Substitution	0:25	2:46
F07 Recovery Aware Adjustment	0:25	3:26
F08 Max Testing and Attempt Planning		3:01 deterministic 4:31 agent
F09 Macro Targets and Adherence	1:11	
F10 Meal Suggestion	1:11	4:54
F11 Coaching Chat with Memory	1:22	
F12 Plan Adherence and Progression Review		5:19
F13 Guided Onboarding	2:05	
F14 Adopt an Existing Programme		6:57
N01 Training Report Export (new)	1:42	
N02 Ask Coach Drawer (new)	1:31	
N03 Light and Dark Themes (new)	2:16	
N04 Guided Tour (new)	0:00	

The timestamps for the GUI demo are counted from the moment the tour starts. Collectively the recordings show both the graphical interface and the command line working, as required. No API key, credential or other secret appears in any recording.

Figure 1. Complete implemented class model, 144 classes.

Every class in the submitted system. Zoom to read; the same image is in the repository at docs/diagrams/png/class-full.png, and the editable UMLet source at docs/diagrams/uxf/class-full.uxf.

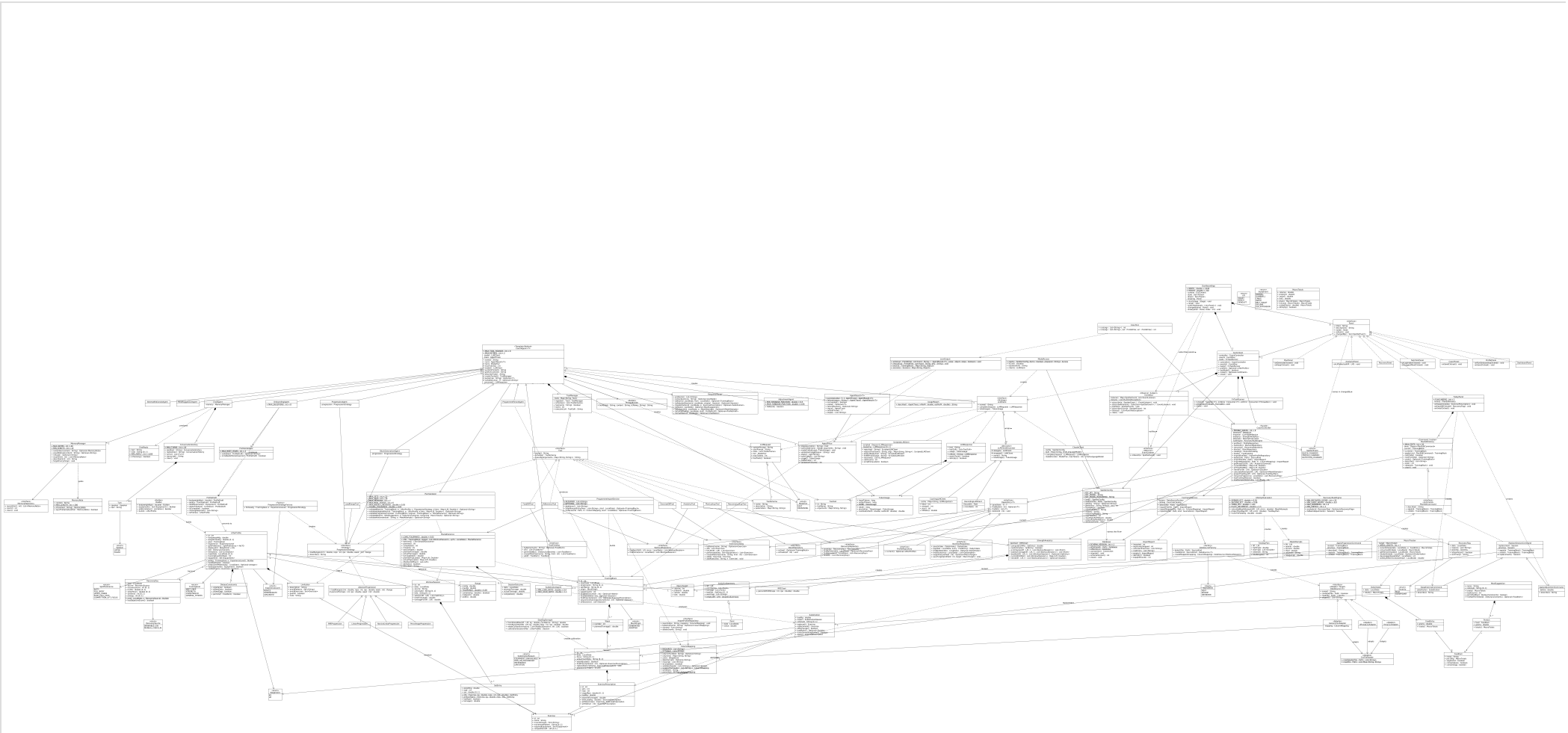


Figure 2. Presentation and application layers, 26 classes.

DashboardApp and the eight Panel implementations, CoachCLI, and the application layer: CoachController as Facade, EventBus as Observer subject, and the PlanEditCommand hierarchy.

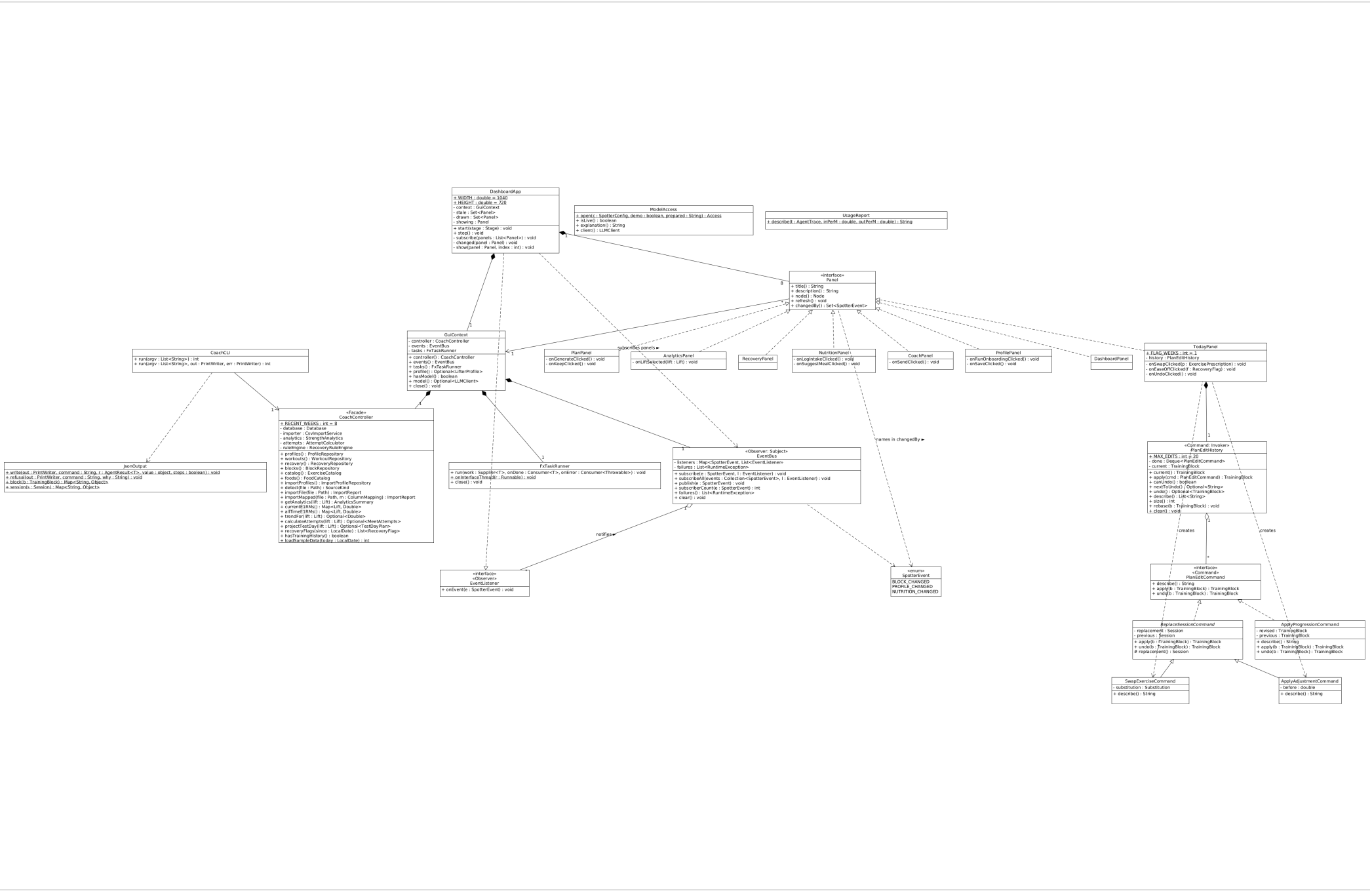


Figure 3. Domain layer, 53 classes.

Entities and the deterministic calculations. Nothing here depends on the agent layer, the database or the interface.

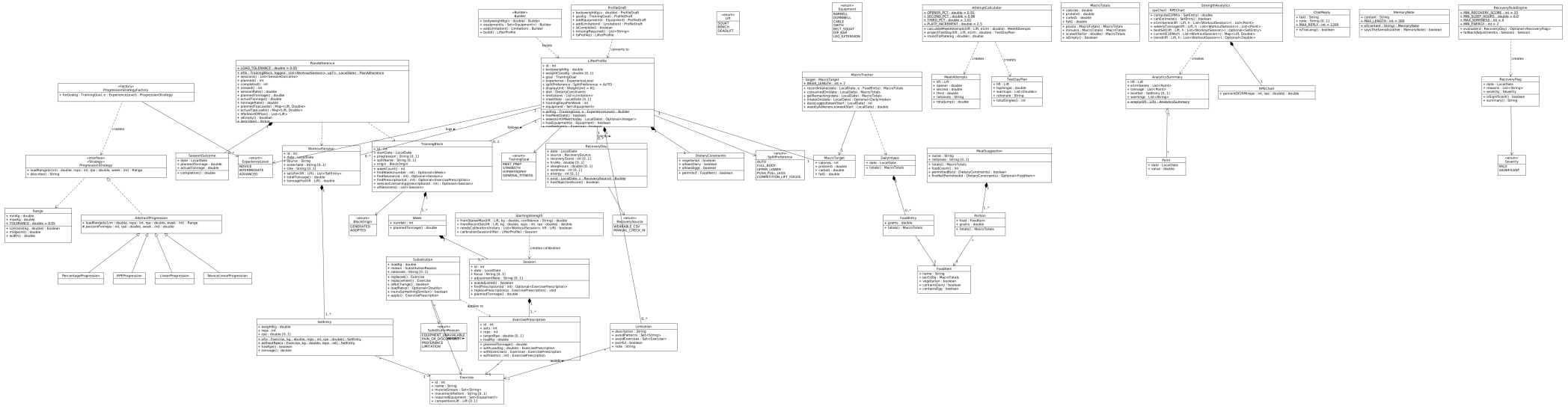


Figure 4. Agent layer, 41 classes.

CoachAgent as Template Method with its nine concrete agents, the LLMClient Strategy with its Decorator chain, the seven tools, and the two validators.

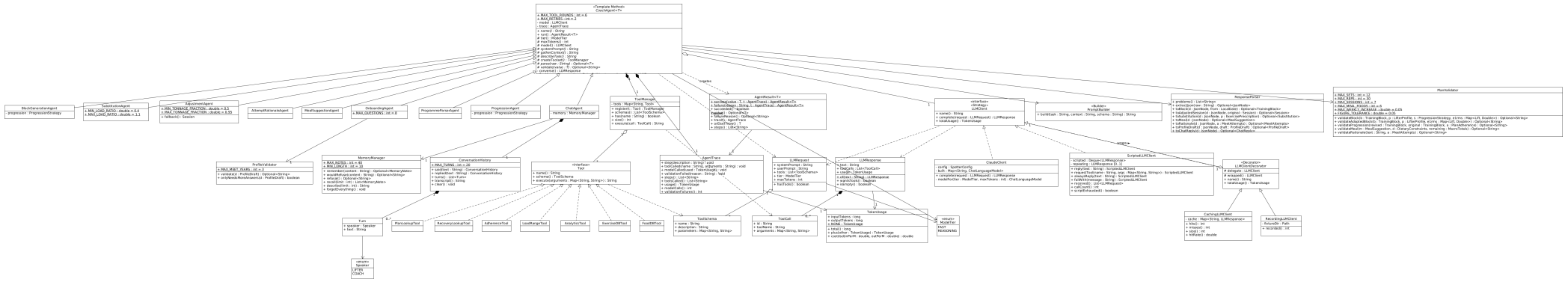


Figure 5. Infrastructure layer, 24 classes.

The DataSource Adapter hierarchy, the repository interfaces, and configuration.

